Serialization and Normalization - Quick Reference

🎯 One-Liner Definitions

Serialization: Converting objects to storable/transmittable format (object → JSON)
Normalization: Organizing data to reduce redundancy and improve integrity (schema design)

📊 Side-by-Side Comparison

Aspect	Serialization	Normalization
What	Process	Data structure
When	Converting data	Designing database
Input	Objects/Arrays	Requirements
Output	JSON/XML/bytes	Tables/relationships
Goal	Store/transmit	Reduce redundancy
Example	dict → JSON	Split into tables

🔄 Serialization in This Project

Serialization Process

```
Python Object
  ↓
model.to_dict()
  ↓
Python dict
  ↓
json.dumps() or jsonify()
  ↓
```

```
JSON string
  ↓
HTTP Response / Database
```

Key Methods

```
# User.to_dict()
{"id": 1, "username": "john", "email": "john@example.com"}

# SchemaModel.to_dict(include_fields=True)
{
  "id": 1,
  "name": "Employee Schema",
  "version": 1,
  "fields": [...]  # Nested serialization
}

# MetadataRecord.to_dict(include_values=True)
{
  "id": 1,
  "schema_id": 1,
  "values": {"name": "John", "age": 30},
  "storage_type": "json"  # or "eav" or "table"
}
```

Files to Check

File	Purpose
`flask_backend/app/models.py`	All `to_dict()` methods
`flask_backend/app/schemas.py`	Marshmallow serialization
`flask_backend/app/routes/*.py`	`jsonify()` calls

Common Operations

```
# Serialize single object
return jsonify(record.to_dict())

# Serialize list
return jsonify([r.to_dict() for r in records])
```

```
# Serialize with filtering
return jsonify(record.to_dict(include_values=False))

# Manual serialization
data = json.loads(raw_json_string)
data = json.dumps(python_dict)
```

🏗️ Normalization in This Project

Three Storage Strategies

1. EAV (Entity-Attribute-Value)

- **When**: Individual field tracking needed
- **Where**: `FieldValue` table
- **Format**: One row per field value
- **Pros**: Type-safe, traceable, flexible
- **Cons**: Many joins, slower queries

```
# EAV Storage
FieldValue(
  record_id=1,
  field_id=1,  # "name" field
  value_text="John"
)
```

2. JSONB (Semi-structured)

- **When**: Bulk data, no field-level tracking
- **Where**: `DataRow` table or `metadata_json` column
- **Format**: Entire row as JSON object
- **Pros**: Fast, simple, flexible schema
- **Cons**: Less typed, harder to validate

```
# JSONB Storage
DataRow(
  record_id=1,
  row_index=1,
```

```
    data={"name": "John", "age": 30, "email": "john@example.com"}
  )
```

3. Traditional (Structured)

- **When**: Fixed schema, relationships
- **Where**: Schema definitions, metadata_records table
- **Format**: Relational columns
- **Pros**: Referential integrity, ACID
- **Cons**: Not flexible, needs migration for schema changes

```
  # Structured Storage
  MetadataRecord(
    id=1,
    schema_id=1,
    asset_type_id=2,
    created_by=1
  )
```

Normalization Levels

Level	Rule	Example
1NF	Atomic values	No lists in columns
2NF	Functional dependency	Each field depends on PK
3NF	No transitive dependency	user_name via FK only

When to Use Each

```
Small single record?
  └─ Use JSONB (metadata_json) ✓ simple

Medium record with tracking?
  └─ Use EAV (FieldValue) ✓ type-safe

Large bulk data?
  └─ Use DataRow table ✓ indexed JSONB

Need relationships?
  └─ Use Foreign Keys ✓ referential integrity
```

🔗 Serialization ↔ Normalization

How They Work Together

```
DESIGN PHASE (Normalization)
  ├─ Design tables: schemas, schema_fields, metadata_records
  ├─ Define relationships: ForeignKey constraints
  └─ Choose storage: EAV vs JSONB vs traditional

DEVELOPMENT PHASE (Both)
  ├─ Store data (Normalization): Insert into normalized tables
  ├─ Serialize data (Serialization): to_dict() for API
  └─ Result: Normalized storage, serialized API

QUERY PHASE (Both)
  ├─ Query database (Normalized): SELECT with JOINs
  ├─ Serialize results (Serialization): model.to_dict()
  └─ Return to client: JSON
```

Example: Complete Flow

```
REQUEST:
POST /metadata
{"schema_id": 1, "values": {"name": "John", "age": 30}}
          ↓
DESERIALIZE:
request.get_json()
{"schema_id": 1, "values": {...}}
          ↓
NORMALIZE:
Create MetadataRecord(schema_id=1, metadata_json={...})
Validate against SchemaModel(id=1)
          ↓
STORE:
INSERT INTO metadata_records (schema_id, metadata_json)
          ↓
QUERY:
SELECT * FROM metadata_records WHERE id=?
JOIN schemas ON metadata_records.schema_id = schemas.id
```

```
          ↓
SERIALIZE:
metadata_record.to_dict()
{"id": 1, "schema_id": 1, "values": {...}}
          ↓
RESPONSE:
200 OK
{"id": 1, "schema_id": 1, "values": {...}}
```

💡 Key Concepts

Serialization Concepts

Concept	Meaning	Example
Serialize	Object → Bytes/String	`to_dict()`
Deserialize	Bytes/String → Object	`json.loads()`
Format	Target representation	JSON, XML, CSV
Encoding	Byte representation	UTF-8, ASCII

Normalization Concepts

Concept	Meaning	Example
Redundancy	Repeated data	Same customer_name in multiple orders
Dependency	One field depends on another	employee_name depends on employee_id
Integrity	Data consistency	ForeignKey constraint
Anomaly	Update/Delete problems	Update name in one place, miss another

---

🔍 Finding Serialization Code

By Operation

Where to find JSON serialization:

- `flask_backend/app/models.py` → `to_dict()` methods

- `flask_backend/app/routes/*.py` → `jsonify()` calls
- `flask_backend/app/schemas.py` → Marshmallow schemas

Where to find deserialization:

- `flask_backend/app/routes/*.py` → `request.get_json()`
- `flask_backend/app/routes/*.py` → `json.loads()`
- Frontend → `JSON.stringify()` / `await response.json()`

By Model

Model	Serialization	Location
User	`to_dict()`	models.py line 15
SchemaModel	`to_dict(include_fields=True)`	models.py line 65
SchemaField	`to_dict()`	models.py line 105
MetadataRecord	`to_dict(include_values=True)`	models.py line 165
DataRow	`to_dict()`	models.py line 225

🔍 Finding Normalization Code

By Strategy

EAV Storage:

- `FieldValue` model in `models.py`
- Stores individual field values
- Type-specific columns: value_text, value_int, value_float, value_bool, value_date, value_json

JSONB Storage:

- `DataRow` table in `models.py`
- PostgreSQL JSONB column for structured data
- `metadata_json` column in MetadataRecord (legacy)

Relationships:

- `ForeignKey` definitions in `models.py`
- `db.relationship()` backref definitions
- Cascade rules: CASCADE, SET NULL

By Problem

"How do schemas avoid migration?"
→ EAV model in `models.py` line 200+

"How is data stored for bulk imports?"
→ DataRow table in `models.py` line 185+

"How are field values typed?"
→ FieldValue with type-specific columns

"How are relationships maintained?"
→ ForeignKey constraints with CASCADE/SET NULL

⚡ Quick Recipes

Serialize Single Record

```
record = MetadataRecord.query.get(1)
return jsonify(record.to_dict())
```

Serialize List

```
records = MetadataRecord.query.all()
return jsonify([r.to_dict() for r in records])
```

Deserialize Request

```
data = request.get_json()
record = MetadataRecord(
    schema_id=data['schema_id'],
    metadata_json=data.get('values')
)
db.session.add(record)
db.session.commit()
```

Handle Multiple Storage Types

```
record.to_dict(include_values=True)
# Automatically returns:
# - values from EAV (FieldValue)
# - OR values from JSONB (DataRow)
# - OR values from metadata_json
```

Join Normalized Tables

```
record = MetadataRecord.query\
    .join(SchemaModel)\
    .filter(SchemaModel.id == 1)\
    .first()

# Access nested objects
schema = record.schema
fields = schema.fields
```

🐛 Common Issues & Fixes

Issue: "Circular Reference" (Serialization)

```
# Problem:
User → Projects → Users (circular)

# Solution:
def to_dict(self, include_projects=False):
    result = {"id": self.id, "name": self.name}
    if include_projects:
        result["projects"] = [p.to_dict(include_users=False) for p in sel
    return result
```

Issue: "Type Lost in JSON" (Serialization)

```
# Problem:
123 (int) serialized → deserialize → "123" (string)

# Solution:
```

```
Use Marshmallow with type validation
schema = UserSchema()
user = schema.load({"id": 123})  # Validates type
```

Issue: "Update Anomalies" (Normalization)

```
# Problem:
UPDATE orders SET customer_name = 'Jane'
WHERE order_id = 5;
// But customer_name is elsewhere, inconsistent!

# Solution:
-- Normalized: Only store customer_id
UPDATE orders SET customer_id = 2 WHERE order_id = 5;
-- Join to get customer_name when needed
```

Issue: "Too Many Joins" (Normalization)

```
# Problem:
SELECT * FROM users
JOIN projects ON users.id = projects.user_id
JOIN tasks ON projects.id = tasks.project_id
// Slow with 1000s of records

# Solution:
Use DataRow with denormalized JSONB for bulk data
SELECT * FROM data_rows WHERE data->>'status' = 'active'
```

📚 Reference Files

File	Contains
`models.py`	All serialization (to_dict) and normalization (EAV, relationships)
`schemas.py`	Marshmallow serialization/validation
`routes/metadata.py`	JSON (de)serialization in endpoints
`routes/schemas_dynamic.py`	Schema normalization operations
`services/*.py`	Complex serialization/normalization logic

🎓 Learning Path

1. **Understand Serialization**
    - Start: `models.py to_dict()` methods
    - Then: `routes/metadata.py jsonify()` calls
    - Practice: Add custom `to_dict()` parameters
2. **Understand Normalization**
    - Start: `models.py` relationships (ForeignKey)
    - Then: EAV model structure
    - Then: JSONB vs relational trade-offs
3. **Apply Together**
    - Serialize: Call `to_dict()` on related objects
    - Normalize: Join tables efficiently
    - Test: Make sure data is consistent

This guide complements **SERIALIZATION_AND_NORMALIZATION_GUIDE.md** with quick lookups and recipes.